GPU-Resident AES-CTR Encryption of Independent Data Pages: Computational Gains and Transfer Limits

Marko Mihajlovic[1]

Abstract: Accelerating AES-CTR with a GPU is useful only when the gains from parallel computation exceed host–device data-movement costs. This study investigates that condition using validated AES-128 CUDA implementations, a native OpenSSL EVP AES-NI baseline, and a batch of independent 1 MiB data pages motivated by columnar storage encryption. Five repeated Tesla T4 sessions show that, for 32 independently encrypted pages (32 MiB), GPU-resident AES-CTR computation achieves a 2.251× speedup relative to the prepared native CPU transformation, whereas the transfer-inclusive prepared GPU pipeline achieves only 0.913×. The corresponding median times are 3.714 ms for the GPU resident kernel, 8.362 ms for OpenSSL transformation, and 9.153 ms for the GPU pipeline. Page-aware GPU execution has approximately the same pipeline throughput as a continuous 32 MiB CTR stream (a 1.007× page-to-continuous ratio), indicating little added cost for independent counter domains in this implementation. In a separate same-runtime comparison, the apparent 32 MiB GPU advantage changes from 5.586× against PyCryptodome to 0.942× against native OpenSSL. The practical opportunity is therefore GPU-resident encryption as a stage within an already device-resident workflow, not general-purpose offloading of host-resident data. That application-level opportunity remains a conditional implication rather than a measured end-to-end application speedup.

Keywords: AES-CTR, GPU-resident encryption, independent data pages, CUDA, AES-NI, OpenSSL, data movement, heterogeneous processing.

1 Introduction

The Advanced Encryption Standard (AES) is widely used to protect stored and transmitted data [1]. Counter (CTR) mode exposes independent block-encryption operations and is amenable to parallel execution [2]. Nevertheless, a GPU is not automatically the fastest place to perform encryption: host processors provide dedicated AES instructions, and moving plaintext and ciphertext across the CPU–GPU boundary consumes time. An application using an accelerator must therefore distinguish the cost of AES computation from the cost of placing data where that computation occurs.

This distinction matters for storage and analytics workloads consisting of many

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

independent data units. The Apache Parquet encryption specification provides a concrete motivation: its `AES_GCM_CTR_V1` mode uses CTR encryption for data pages and GCM protection for other encrypted modules. Rather than claim support for a complete Parquet file, we study a controlled, Parquet-inspired workload of independent 1 MiB pages with a distinct 96-bit nonce and a 32-bit counter initialized to one for each logical page. The study does not implement file framing, authenticated metadata, key management, compression, or Parquet interoperability. CTR encryption alone does not provide authentication.

The central question is not simply whether a GPU can execute AES quickly, but *where a measured advantage exists and what must change in data placement for an application to exploit it*. A GPU-resident input can be encrypted without round-trip payload movement. A CPU-resident input that must return to host memory after GPU encryption requires two transfers. These are distinct workflows and must not be described by a single generic "GPU speedup."

The study makes three bounded contributions. First, it establishes a same-runtime comparison with native OpenSSL EVP AES-NI, correcting the interpretation of earlier comparisons that used PyCryptodome as their host reference. Second, it quantifies how a GPU compute-only advantage for 32 MiB CTR is lost at a prepared transfer-inclusive boundary. Third, it validates independent nonce/counter domains per page and shows that their GPU processing cost is close to the cost of a continuous stream of equal length. The contribution is a measured execution-placement distinction and a reproducible page-aware implementation, not a new AES primitive or a demonstrated complete storage-engine speedup.

Previously recorded measurements on GTX 1650 Ti, T600 and Tesla T4 are retained as historical, implementation-specific evidence. New same-runtime T4 CPU/GPU and page-batching sessions are presented separately; their observations are not pooled with the earlier data. Section 3 defines the measurement boundaries, Section 4 presents the positive and negative results together, and Sections 5 and 6 discuss the practical conditions and remaining work.

2 Related Work and Research Gap

Early contributions by Manavski [3] and Harrison and Waldron [4] established the feasibility of AES execution on programmable GPU architectures. Subsequent work investigated how parallel processing, memory organization, and architecture-specific optimization affect encryption throughput. The available parallelism depends partly on the encryption mode: ECB permits independent block transformations, while CTR generates keystream from distinct counter values [2]. Both expose independent work for GPU threads when buffers are sufficiently large. ECB is unsuitable for protecting general structured data and serves here only as a

controlled performance reference.

Iwai et al. [5] examined thread granularity, memory organization, and the interaction between computation and data transfers, including overlapping transfers with computation. Their CUDA implementation achieved approximately 35 Gbit/s for device-side AES processing and approximately 22.5 Gbit/s for transfer-aware execution. These results demonstrate that the execution boundary and transfer strategy affect the reported encryption rate. Wang and Chu [6] investigated lookup-table-based AES implementations and GPU execution strategies. Tezcan [7] examined shared-memory optimization and bank-conflict avoidance, reporting AES-128 encryption throughput of 878.6 Gbit/s on an NVIDIA RTX 2070 Super.

More recent work has explored bit-sliced cipher representations. A 2024 study [8] reported high-performance AES-CTR and AES-ECB execution on desktop and embedded NVIDIA platforms. Together, these studies demonstrate the computational capabilities of optimized GPU implementations. Their reported rates must nevertheless be interpreted within their respective measurement boundaries: device-side encryption throughput does not directly specify the rate at which an application can receive, encrypt, and deliver data. It also does not establish whether offloading improves performance relative to the host processor available in a particular system.

That comparison depends on the CPU implementation. Dedicated instructions such as AES-NI reduce the cost of AES transformations [9] and provide a hardware-assisted baseline for evaluating GPU offloading. Lee et al. [10] demonstrated that conclusions about heterogeneous acceleration depend strongly on the optimization level of the implementations being evaluated. Their findings support using an appropriately accelerated CPU reference when deciding whether GPU offloading offers an additional processing benefit. Accordingly, comparing GPU encryption with conventional unaccelerated software cannot establish an advantage over the host's hardware-assisted AES path. The reference must represent the execution path against which an offloading decision will actually be made, and the comparison remains specific to the implementations evaluated.

Data location further determines which costs enter that decision. Gregg and Hazelwood [11] showed that CPU–GPU performance evaluation requires explicit consideration of data movement. An advantage for device-resident input may shrink when both input and output cross the host–device boundary. CUDA guidance recommends minimizing unnecessary transfers, using appropriate synchronization, and considering overlap between communication and computation [12, 13]. The transfer-aware AES work of Iwai et al. [5] demonstrates that this concern is already established in GPU cryptographic processing.

For storage-oriented workloads, the Apache Parquet encryption specification

supports the extttAES_GCM_CTR_V1 construction, in which encrypted data pages motivate independently initialized CTR streams [14]. Independent pages expose parallel work but require a distinct nonce/counter domain for each page. Unlike a single contiguous CTR stream, page-aware execution must map each GPU thread to a page-local counter without counter reuse. GPU-assisted GCM research [15, 16] is related, but accelerating AES-CTR alone does not provide authentication or a complete file-format implementation.

Assessing whether an observed advantage is repeatable also requires control of experimental variability. Mytkowicz et al. [17] showed that experimental conditions can introduce systematic bias and alter performance results. Kalibera and Jones [18] emphasized the importance of accounting for variability across multiple levels of experimental replication, while Hoefler and Belli [19] discussed scientifically defensible performance evaluation of parallel computing systems. These contributions provide a methodological basis for determining whether an acceleration is repeatable under the conditions tested. These studies motivate reporting repeated observations and their execution context rather than relying on the highest throughput measurement. For cryptographic workloads, functional correctness must additionally be verified before timings are accepted as evidence of encryption performance.

These studies motivate correctness-gated measurements, session-level variability, and explicit definitions of what is timed. The unresolved practical question is whether GPU computation remains attractive against a native hardware-accelerated CPU when independently encrypted pages and host–device transfers are measured. We address this with a same-runtime Tesla T4 study, while retaining a compact historical cross-platform comparison as context rather than treating it as statistically paired evidence.

3 Methodology

The central study measures AES-128-CTR encryption of host-resident continuous buffers and groups of independently encrypted 1 MiB pages on a Linux x86-64 Google Colab instance with an NVIDIA Tesla T4 (compute capability 7.5). The CPU comparator is native OpenSSL 3 EVP with AES-NI capability enabled; PyCryptodome serves as an implementation sensitivity control. The CUDA implementation retains the original AES rounds and executes 128 threads per block. All comparisons in the main study use the same live runtime, deterministic input material, and ciphertext validation.

Earlier benchmarking covered a GeForce GTX 1650 Ti/i5-10300H on Windows, an NVIDIA T600/i7-11850H on Windows, and a Tesla T4/Xeon under Linux. These records remain historical context only: they used PyCryptodome host

intervals that are not equivalent to the new native OpenSSL boundary and cannot be paired with the later campaigns. An independent Oracle Cloud Neoverse-N1 ARM64 CPU-only control was also collected; it informs the interpretation of library/capability sensitivity but contributes no GPU speedup ratios. Platform and library versions and source hashes are recorded with the released experiment artifacts.

3.1 Native CPU Baseline and Same-Runtime T4 Validation

The historical three-platform study used a hardware-assisted PyCryptodome host path. A later, independent T4 study compared that path, a native OpenSSL 3 EVP AES-128 implementation, and the original validated CUDA baseline on the same runtime allocation. The five sessions used seeds 2003–2007, five warm-ups and 20 measurements per size and mode; buffers ranged from 1 KiB to 32 MiB. Results from this newer study are paired only within its own sessions.

Native *CPU transform time* was measured inside C around encryption with prepared contexts/output; it excludes the Python-to-C call, allocation and cipher setup. GPU *resident time* is a CUDA-event compute interval with prepared device buffers. GPU *prepared pipeline time* includes payload host-to-device transfer, kernel execution, device-to-host transfer and synchronization, but excludes GPU allocation, staging, key expansion and output conversion. These boundaries are useful for determining whether prepared GPU processing repays transfer cost, but they do not provide identical wrapper costs or full application latency. CPU operation-envelope measurements include additional setup/allocation/copying and are retained only as asymmetric diagnostics.

3.2 Validation, Timing, and Statistical Treatment

The experiments distinguish three operational questions. First, a resident-kernel interval tests the encryption computation achievable when the payload has already been placed in device memory. Second, a prepared-pipeline interval tests encryption when the plaintext begins in host memory and the encrypted payload must return to the host; this interval includes synchronous host–device transfers and GPU completion. Third, a native OpenSSL transform interval supplies an optimized, hardware-assisted host reference without including buffer allocation or high-level language binding costs. We deliberately avoid labelling this collection as a symmetric application-level benchmark. The independent-page test does not include a GPU implementation of page-level metadata protection, authentication, or a Parquet writer.

Every configuration uses deterministic, matched inputs and a correctness gate before recorded timing. Timed ciphertext is also checked against an independently computed reference outside the measurement window. An invalid counter

range, repeated nonce within a logical batch, or output mismatch invalidates the observation. GPU launches operate on preallocated buffers; the prepared pipeline includes both payload transfer directions, but does not include the prior creation of pinned buffers or encryption-key expansion. Consequently, a difference between pipeline and resident sample medians is not interpreted as a pure PCIe transmission measurement.

For a buffer of B bytes and duration t seconds, throughput is $B/(10^6 t)$ in decimal MB/s. Within a session, the primary speedup estimator is the ratio of median CPU and GPU durations for the same configuration; five such session estimates are summarized by their median and range. Matched per-repetition ratios are retained as a separate estimator rather than silently mixed with ratios of medians. No warm-up durations enter the summary, and no adverse observations or outliers are deleted. Since five sessions are repeated executions on one cloud allocation rather than independent hardware samples, these statistics describe consistency on the tested system and do not support population-level confidence claims.

3.3 Independent-Page CTR Batch Experiment

A separate T4 campaign encrypted $N \in \{1, 4, 16, 32\}$ independently managed 1 MiB pages, for total sizes of 1, 4, 16 and 32 MiB. Each page used a separate 96-bit synthetic test nonce concatenated with a 32-bit big-endian initial counter of one. The counter is advanced *within* each page and never carried into another page's nonce. The page-aware CUDA kernel assigns blocks to their corresponding page and encrypts all pages in one launch using the unchanged historical AES round implementation. A native OpenSSL EVP path processes the page contexts sequentially on one calling CPU thread.

An equal-size continuous CTR stream forms the control. For each size and repetition the page and continuous controls share plaintext and key but use distinct nonce domains. The experiment validates ciphertext independently against a CPU reference, including published vectors, page boundaries, counter carry and overflow guards. Five T4 sessions (seeds 2003–2007) each include five warm-ups and 20 measurements for all four batch sizes, both layouts and three scheduled measurement paths. This yields 2,400 measured observations plus 600 warm-ups. Actual execution order, input digests, source hashes, software versions and session identity are retained. All reported speedups use the CPU duration divided by GPU duration: values greater than one favor the GPU. Five session medians, not pooled individual repetitions, form the primary descriptive replication units. The study is synthetic and measures batch completion rather than application request latency.

4 Results

4.1 Historical Cross-Platform Context

The original 32 MiB AES-128-CTR campaign reported median host-relative prepared-pipeline speedups of 0.935× on the GTX 1650 Ti, 1.354× on the T600, and 4.761× on the Tesla T4. Those measurements used PyCryptodome host calls and include differences in software versions, CPU dispatch, platform scheduling and API scope. The result establishes that measured offload advantage varied between the three complete configurations; it does *not* establish that the T4 outperforms a native OpenSSL CPU transform. The updated same-runtime comparison below resolves that ambiguity. Earlier ECB, small-buffer and throughput observations remain in the reproducibility archive and are not used to infer page-workload effects.

Table 1 preserves the historical 32 MiB mode comparison, including resident and transfer-inclusive outcomes. It is presented for context, not as a pooled analysis with the later T4/OpenSSL campaign. The T600 and T4 CTR configurations favored the GPU under their original PyCryptodome-based boundaries; the GTX result was near parity. Those historical results therefore motivate–rather than replace–the optimized-host experiment.

Table 1
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

System	Mode	R	P	P range	$P > 1$
GTX	ECB	3.091	0.761	[0.722, 0.818]	0/5
GTX	CTR	3.693	0.935	[0.918, 1.008]	1/5
T600	ECB	2.215	1.118	[0.659, 1.144]	3/5
T600	CTR	2.666	**1.354**	[1.129, 1.391]	**5/5**
T4	ECB	2.207	0.893	[0.868, 0.901]	0/5
T4	CTR	11.758	**4.761**	[4.691, 5.021]	**5/5**

Bold pipeline medians and counts indicate acceleration in all five sessions. Resident execution is faster in all five sessions for every row.

Figure 1 provides the absolute throughput counterpart to those ratios. The differences between resident and prepared-pipeline rates illustrate why a peak kernel figure alone does not determine a complete host-to-host encryption rate.

4.2 Oracle Cloud ARM64: CPU Implementation and Envelope Controls

On the independent Oracle Cloud ARM64 host, the five-session 32 MiB AES-128-CTR study produced approximately 4.03 GB/s of native OpenSSL-

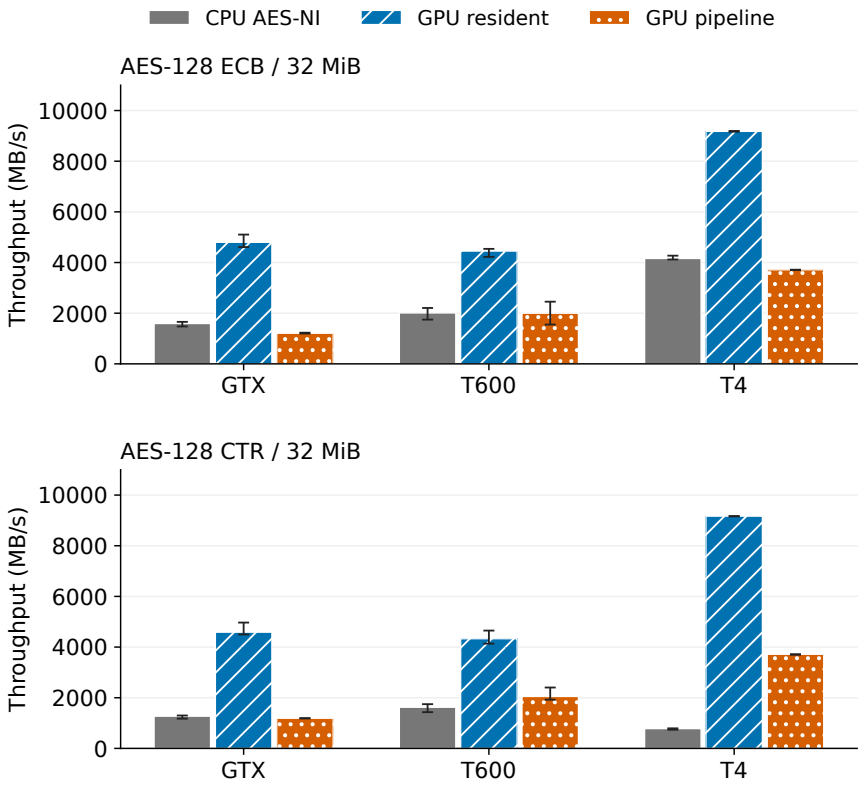


Fig. 1 — Historical 32 MiB AES-128 throughput for three complete configurations. CPU references use the original PyCryptodome paths; resident GPU timing excludes host–device transfers, whereas the prepared pipeline includes synchronous payload transfers. These data are not paired with the later OpenSSL-controlled campaign.

default transform-only throughput, compared with approximately 209 MB/s for the PyCryptodome path and 218 MB/s for the `OPENSSL_armcap=0` control. These values describe the specific library implementations and timed boundaries; they are not GPU speedups. The default OpenSSL path was approximately 19.3 times faster than the recorded PyCryptodome transform path at this size, with the important qualification that Python/FFI and C-native interval definitions differ.

In a separate host-side envelope experiment, a single worker achieved approximately 1.06 GB/s at 32 MiB, while two concurrent workers reached approximately 2.03 GB/s in aggregate, or about 1.92 times the single-worker aggregate rate. These campaigns are not paired per-repetition observations and their envelope boundary differs from the 4.03 GB/s native transform interval. The contrast demonstrates that buffer lifecycle and concurrency must be considered before extending cryptographic core throughput to a complete service. No Oracle result is used to calculate a speedup against the Tesla T4 or other GPU system.

4.3 Revisiting the Host Baseline on the Tesla T4

The historical 4.761× T4 CTR speedup in the preceding tables is an *earlier PyCryptodome-relative result*, not a speedup relative to an optimized native OpenSSL implementation. In a new five-session T4 campaign, 32 MiB AES-128-CTR yields a 5.586× median session ratio against PyCryptodome but only 0.942× against native OpenSSL when the GPU prepared pipeline is the denominator. The OpenSSL transform throughput is approximately 3.874 GB/s and the GPU prepared-pipeline throughput is approximately 3.646 GB/s. When only GPU resident computation is considered, the native OpenSSL CPU/GPU ratio is 2.299×. Thus the improved CPU baseline reverses the prepared-pipeline ranking, without implying that the historical measurements were incorrect.

4.4 Independent Pages: A Compute Advantage That Does Not Survive Transfers

Table 2 gives the prespecified native OpenSSL transform to prepared CUDA pipeline time ratios in the page-batching study. Every reported ratio is below unity across all five sessions. The gap narrows at larger batch sizes but does not reverse direction.

The most informative case is a batch of 32 independent pages. The median CPU native transform duration is approximately 8.362 ms, compared with 3.714 ms for one GPU-resident kernel and 9.153 ms for the prepared GPU pipeline. Table 3 separates the measurements. The compute-only ratio is 2.251× in favor of the GPU, whereas the transfer-inclusive ratio is 0.913× and favors the CPU. Both directions are consistent across all five sessions. The observed H2D-plus-D2H component is about 5.418 ms. Medians of nested or separately timed components

Table 2
Tesla T4 Phase 2: median of five session-level CPU OpenSSL transform / GPU prepared-pipeline ratios. Values below one favor the CPU at this boundary.

Total payload	Continuous CTR	Independent 1 MiB pages
1 MiB	0.5437	0.5404
4 MiB	0.6948	0.7137
16 MiB	0.8163	0.8240
32 MiB	0.8963	0.9125

cannot be added as if they described a single observation.

Table 3
Tesla T4 Phase 2, 32 independent 1 MiB pages: separately summarized timings (five session medians).

Measurement boundary	Approximate time (ms)
Native OpenSSL CPU transform	8.362
CUDA resident kernel	3.714
GPU H2D + D2H transfers	5.418
CUDA prepared pipeline, including transfers and synchronization	9.153
GPU compute-only speedup versus CPU transform	2.251×
GPU prepared-pipeline speedup versus CPU transform	0.913×

The page-aware GPU prepared pipeline shows almost no relative penalty against a single continuous CTR stream of the same size: its page/continuous throughput ratios are approximately 1.0007× at 4 MiB, 1.0068× at 16 MiB and 1.0066× at 32 MiB. These sub-percent differences should not be described as a robust optimization gain. They support the narrower observation that independent per-page counter handling does not impose a large measured GPU pipeline overhead in the evaluated implementation. This is distinct from demonstrating an advantage from batching over sequential per-page GPU launches, which was not the control experiment.

5 Discussion

5.1 The Specific Favorable Case: Data Already on the GPU

The positive result is precise: for 32 independently encrypted 1 MiB pages, one CUDA resident AES-128-CTR launch takes approximately 3.714 ms, whereas native OpenSSL EVP requires approximately 8.362 ms for the prepared sequential

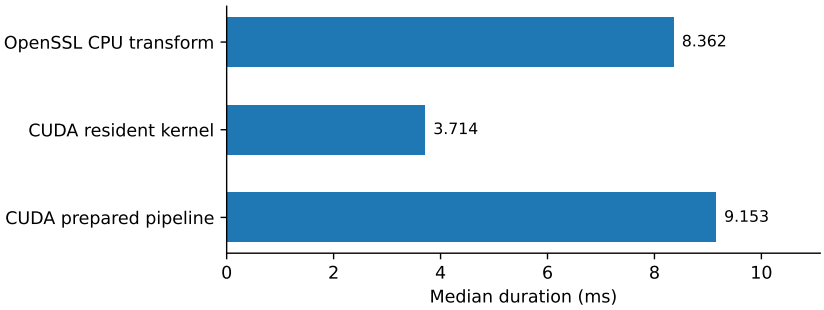


Fig. 2 — Observed 32 MiB independent-page timing at three explicitly different boundaries. Each bar represents the median of five session medians; the GPU resident bar excludes host–device transfers, while the prepared-pipeline bar includes them. The figure compares measured intervals, not identical application-level API calls.

CPU transformation. The measured 2.251× ratio makes GPU execution a credible compute-stage candidate when plaintext is already GPU-resident and the encrypted result is needed on the device for another processing stage. This is not a claim that an ordinary host-memory encryption service becomes 2.251× faster: the resident interval excludes data placement, GPU allocation and preparation, and a complete multi-stage application has not been benchmarked.

Potential workload classes include GPU-native analytics pipelines, pipelines that prepare columnar buffers on the GPU before encryption, and device-side processing that passes the resulting ciphertext to a later GPU operation. The common factor is *avoided payload round trips*, not the mere presence of a GPU. Such placement can be attractive only if input data originate on the device or would need to move to it for other work anyway, and if the required output does not have to return immediately to the CPU. Where plaintext starts in host RAM and ciphertext is consumed by a host process, the tested prepared pipeline takes approximately 9.153 ms versus 8.362 ms for CPU transformation: the CPU is faster at these declared boundaries.

5.2 A Measured Data-Movement Budget, Not an Application Speedup

The 32 MiB prepared GPU pipeline is approximately 0.791 ms slower than the prepared CPU transformation, based on the displayed session-level median durations. This difference describes the margin to parity for the measured configuration, not an empirically achieved optimization. The approximately 5.418 ms transfer component demonstrates that data movement is large relative to AES kernel execution; its separately reported median is not directly substitutable into a single-iteration latency equation. Avoiding transfers could make GPU-

resident processing competitive, but the exact crossover requires a new workload-level experiment including data preparation, input residency, output placement, any subsequent consumer, and buffer lifetime.

The ratio of CPU operation-envelope to GPU prepared-pipeline time is not a symmetrical end-to-end comparison: CPU allocation and setup are included, whereas GPU allocation, key upload, pinned-memory staging and final conversion are excluded. It must not be used to claim an application-level speedup. A second caution is that native OpenSSL EVP timings exclude the Python wrapper, unlike parts of the GPU path. The present study therefore provides a well-defined prepared-workload comparison rather than a common application API benchmark.

5.3 Independent Pages and Practical Storage Workloads

Correct independent CTR processing is more than slicing a continuous stream: each logical page needs its own nonce/counter domain, and the counter must not cross the domain boundary. The page-aware CUDA implementation verifies these conditions and processes a batch in one launch. At 32 MiB, the measured GPU pipeline throughput differs by less than 1% from the continuous control; hence handling page boundaries does not impose a large penalty in this specific kernel. The observation does not imply that page batching is faster than separately launching the pages: no separate-per-page launch baseline was included, and the CPU processes the pages sequentially.

The Parquet `AES_GCM_CTR_V1` design [14] motivates the page geometry, but the experiment is not a Parquet writer, reader or encryptor. Format metadata, AAD, authentication, key management, compression and nonce-generation rules remain absent. Raw CTR encryption is malleable and provides no integrity protection. Practical storage deployment therefore requires a complete format-compliant and authenticated workflow; AES-CTR throughput alone cannot support security or end-to-end speedup claims about such a system.

5.4 Why the CPU Baseline Matters

The historical 4.761× T4 result used a PyCryptodome-based host interval. The fresh T4 campaign measured 5.586× against PyCryptodome but only 0.942× against native OpenSSL at the prepared GPU boundary. The native implementation makes the CPU substantially more competitive. Because library versions, API overhead and timing boundaries differ, this is evidence of baseline sensitivity rather than a proof that the old measurement was invalid. Historical GTX 1650 Ti and T600 findings remain useful for showing diversity across complete configurations but cannot substitute for a same-runtime OpenSSL-controlled study on those machines.

The CPU-only Oracle Neoverse-N1 observations offer an independent check

on baseline sensitivity, but they cannot be ranked against T4 GPU times. Their default OpenSSL, PyCryptodome, and capability-masked implementations have distinct interfaces and dispatch options; the observed gap is not an isolated estimate of an AES instruction's effect.

5.5 Generalizability, Constraints and Future Work

The relevant engineering decision is whether a specific pipeline can retain plaintext and ciphertext on the GPU while maintaining its functional and security requirements. The following hypotheses are actionable but untested: (1) a GPU-first columnar transformation followed by resident encryption could avoid round-trip payload transfers; (2) encryption before another GPU-resident consumer could make cipher output placement beneficial; and (3) pipelined transfers or batch reuse might reduce the effective movement cost for host-resident data. None is demonstrated by the present one-launch benchmarks.

The next experiment should implement an actual producer–encrypt–consumer CUDA pipeline, with device-resident buffers, an explicit encrypted-output consumer, correctness validation, and a CPU counterpart completing the same functional work. It should account for input acquisition, context/key and nonce-table setup, GPU allocation, transfers actually needed, synchronization and output disposition. A separate one-page-at-a-time GPU launch baseline would isolate the genuine *batching* effect from the present continuous CTR control. Format-level work would integrate Parquet's full encryption rules, authenticated metadata, and key management, then report file or query throughput and latency. AES-GCM, AES-256, overlapping streams and multiple simultaneous requests are further experiments, not consequences of AES-128-CTR results.

The present measurements use one CUDA implementation and a single-threaded native CPU comparator, five sessions, synthetic data and a shared cloud platform. Kernel-level and transfer timings use different instrumentation; GPU scheduling, memory placement, host contention and CPU dispatch affect results. No instruction-level trace was captured. The work neither ranks all available CPUs/GPUs nor estimates a universal cross-platform crossover.

6 Conclusion

The experiments identify a specific opportunity rather than a blanket GPU advantage: with 32 independent 1 MiB pages, Tesla T4 AES-128-CTR resident execution is approximately 2.251× faster than native OpenSSL EVP transformation at the measured compute boundaries. Once the prepared GPU path includes synchronous host–device transfers and synchronization, that ratio falls to 0.913× and the CPU wins in all five sessions. A page-aware kernel with independent

counter domains has approximately the same GPU pipeline throughput as a continuous stream of equal size, but this does not establish an acceleration benefit over separate per-page launches.

A second key finding is baseline sensitivity: the new T4 study reports a 5.586× prepared-pipeline ratio relative to PyCryptodome but 0.942× relative to native OpenSSL. Consequently, library choice and measurement boundaries are essential to any defensible GPU acceleration claim. Earlier three-platform measurements remain historical evidence tied to their original host implementation, not proof of an optimized-CPU advantage.

An independent Oracle Cloud ARM64 CPU-only control further establishes implementation sensitivity outside x86: at 32 MiB AES-128-CTR, native OpenSSL-default transform throughput was approximately 4.03 GB/s, versus 209 MB/s for the recorded PyCryptodome path, while aggregate operation-envelope throughput increased from approximately 1.06 GB/s with one worker to 2.03 GB/s with two. These differently bounded observations are not compared directly with GPU measurements.

The practical candidate is an application that already keeps its input and subsequent ciphertext consumer on the GPU; in such a pipeline encryption can potentially exploit device-resident computation without paying a full payload round trip. This is an engineering hypothesis supported by the isolated resident result, not an established application speedup. Future work should measure a complete GPU-native producer–encrypt–consumer pipeline against an equivalent CPU workload and separately test one-page-per-launch overhead, Parquet-compliant authenticated storage processing, and overlapping data movement. The principal finding is therefore a reproducible boundary between GPU computational gain and transfer-limited execution, together with a validated independent-page workload for investigating that boundary.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023. URL https://doi.org/10.6028/NIST.FIPS.197-upd1.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001. URL https://doi.org/10.6028/NIST.SP.800-38A.

[3] Svetlin A. Manavski. CUDA compatible GPU as an efficient hardware accelerator for AES cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, pages 65–68, 2007. doi: 10.1109/ICSPC.2007.4728256.

[4] Owen Harrison and John Waldron. AES encryption and decryption on the GPU. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-a

nd-decryption-gpu.

[5] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of AES encryption on CUDA GPU. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[6] Canhui Wang and Xiaowen Chu. GPU accelerated AES algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[7] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[8] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of AES-CTR and AES-ECB bit-sliced implementation on GPUs. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[9] Intel Corporation. Intel advanced encryption standard instructions (AES-NI), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100X GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[11] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate CPU vs. GPU performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[12] NVIDIA Corporation. CUDA C++ best practices guide, n.d. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[13] Mark Harris. How to overlap data transfers in CUDA C/C++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[14] Apache Parquet Project. Parquet Modular Encryption. Apache Parquet file format documentation. URL https://parquet.apache.org/docs/file-format/data-pages/encryption/. Accessed: 2026-10-27.

[15] Georg Schönberger and Jürgen Fuß. GPU-assisted AES encryption using GCM. In *Communications and Multimedia Security*, volume 7025 of *Lecture Notes in Computer Science*, pages 178–185. Springer, 2011. doi: 10.1007/978-3-642-24712-5_16.

[16] JaeSeok Lee, DongCheon Kim, and Seog Chung Seo. Parallel implementation of GCM on GPUs. *ICT Express*, 11(2):310–316, 2025. doi: 10.1016/j.icte.2025.01.006.

[17] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[18] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.

[19] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the*

International Conference for High Performance Computing, Networking, Storage and Analysis, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.